

Dependency Injection und Hosting

ASP.NET Core Integration

Geef.Sdk integriert sich nahtlos in das Microsoft-DI-System (`Microsoft.Extensions.DependencyInjection`).

Einfache Registrierung

```
// Program.cs
builder.Services.AddGeefPipeline<string>(pipeline =>
{
    pipeline
        .UseGrounding(new MyGrounding())
        .UseExecution(new MyExecution())
        .AddReviewer(new MyReviewer())
        .UseFinalizer(new MyFinalizer());
});
```

Der `GeefPipelineRunner<string>` wird als Singleton registriert und kann überall injiziert werden:

```
public class MyController(GeefPipelineRunner<string> pipeline) : ControllerBase
{
    [HttpPost("generate")]
    public async Task<IActionResult> Generate([FromBody] string input, CancellationToken ct)
    {
        var result = await pipeline.RunAsync(input, ct);
        return Ok(new { result.Output, result.TotalIterations, result.RunId });
    }
}
```

Registrierung mit Service-Provider

Wenn Provider selbst aus dem DI-Container aufgelöst werden:

```

builder.Services.AddSingleton<MyGrounding>();
builder.Services.AddSingleton<MyExecution>();
builder.Services.AddSingleton<SecurityReviewer>();
builder.Services.AddSingleton<StyleReviewer>();
builder.Services.AddSingleton<MyFinalizer>();

builder.Services.AddGeefPipeline<CodeResult>((sp, pipeline) =>
{
    pipeline
        .UseGrounding(sp.GetRequiredService<MyGrounding>())
        .UseExecution(sp.GetRequiredService<MyExecution>())
        .AddReviewer(sp.GetRequiredService<SecurityReviewer>())
        .AddReviewer(sp.GetRequiredService<StyleReviewer>())
        .UseFinalizer(sp.GetRequiredService<MyFinalizer>())
        .UseMiddleware(new TimeoutMiddleware
        {
            DefaultTimeout = TimeSpan.FromMinutes(5)
        })
        .AddEventSink(new LoggingEventSink(
            sp.GetRequiredService<ILogger<GeefPipelineRunner<CodeResult>>>()));
});

```

Mehrere Pipeline-Typen

Jeder **TOutput**-Typ erzeugt eine eigene Singleton-Registrierung:

```

// Pipeline für Code-Generierung
builder.Services.AddGeefPipeline<CodeResult>((sp, p) => { /* ... */ });

// Pipeline für Dokumentations-Generierung
builder.Services.AddGeefPipeline<DocumentResult>((sp, p) => { /* ... */ });

// Pipeline für Test-Generierung
builder.Services.AddGeefPipeline<TestResult>((sp, p) => { /* ... */ });

```

```

public class OrchestrationService(
    GeefPipelineRunner<CodeResult> codePipeline,
    GeefPipelineRunner<DocumentResult> docPipeline,
    GeefPipelineRunner<TestResult> testPipeline)
{
    public async Task<FullResult> GenerateAsync(string spec, CancellationToken ct)
    {
        var code = await codePipeline.RunAsync(spec, ct);
        var docs = await docPipeline.RunAsync(code.Output.SourceCode, ct);
        var tests = await testPipeline.RunAsync(code.Output.SourceCode, ct);
        // ...
    }
}

```

Lifetime und Thread-Sicherheit

- `GeefPipelineRunner<T>` wird als **Singleton** registriert
- Der Runner ist **thread-safe** — mehrere gleichzeitige `RunAsync()`-Aufrufe sind sicher
- Jeder `RunAsync()`-Aufruf arbeitet mit eigenen Snapshots (Immutable Context)
- Es gibt keine gemeinsamen veränderlichen Zustände zwischen Runs

Integration mit OpenTelemetry

```

builder.Services.AddOpenTelemetry()
    .WithTracing(tracing =>
    {
        tracing
            .AddSource("Geef.Sdk") // GEEF-Spans registrieren
            .AddAspNetCoreInstrumentation()
            .AddHttpClientInstrumentation()
            .AddOtlpExporter();
    });

```

Damit erscheinen GEEF-Spans (`geef.pipeline.run`, `geef.grounding`, `geef.execution`, `geef.evaluation`, `geef.iteration`, `geef.finalize`) im Tracing-Backend (Jaeger, Zipkin, etc.).